

# Financial Time Series Analysis

## Lecture 12

- ▶ TA Session: Friday, April 30, 11:00am-12:00pm
- ▶ TA Session: Monday, May 3, 7:30-8:30pm

## Outline

- ▶ Final Exam, Monday, May 10, 10:00am (EDT)
- ▶ Cointegration
  - ▶ Granger-Engle
  - ▶ Johansen
  - ▶ Phillips-Ouliaris
- ▶ State Space Models

# Cointegration

- ▶ There are two major approaches to determining the presence of cointegration of two or more time series:

1. Granger-Engle
2. Johansen

- ▶ **Granger-Engle Method**

Represent one time series as a linear combination of the others:

$$X_{jt} = \sum_{i \neq j} \beta_i X_{it} + \epsilon_t.$$

- ▶ Fit using least squares and test whether the residuals are stationary using Dickey-Fuller
- ▶ If the residuals are stationary, then the cointegrating vector is  $(\beta_1, \dots, \beta_{j-1}, -1, \beta_{j+1}, \dots, \beta_m)$ . There can be several cointegrating vectors.

## Cointegration: Johansen 1

- ▶ We begin with a VAR(1) process and later generalize to an AR(p) process.
- ▶ We assume that the process is  $\mathcal{I}(1)$ . If it is  $\mathcal{I}(0)$ , then the original series are all stationary, and there is no substance to cointegration.
- ▶ If the process is  $\mathcal{I}(k)$ ,  $k \geq 2$ , then we difference  $k - 1$  times to bring it back to  $\mathcal{I}(1)$ . Let

$$\mathbf{X}_t = \mathbf{a} + \mathbf{A}\mathbf{X}_{t-1} + \boldsymbol{\epsilon}_t.$$

- ▶ Subtract  $\mathbf{X}_{t-1}$  from both sides to obtain

$$\nabla \mathbf{X}_t = \mathbf{a} + (\mathbf{A} - \mathbf{I})\mathbf{X}_{t-1} + \boldsymbol{\epsilon}_t.$$

## Cointegration: Johansen 2

- ▶ Note  $\mathbf{A} - \mathbf{I}$  cannot be full rank. If it were then

$$(\mathbf{A} - \mathbf{I})^{-1}(\nabla \mathbf{X}_t - \mathbf{a} - \epsilon_t) = \mathbf{X}_{t-1},$$

which would be stationary, contradicting  $\mathbf{X}_t$  is  $\mathcal{I}(1)$ .

- ▶ Now if each component of  $\mathbf{X}_t$  is  $\mathcal{I}(1)$ , then  $\nabla \mathbf{X}$  is stationary. Thus the right side of the equation must also be stationary.

## Cointegration: Johansen 3

- ▶ But  $(\mathbf{A} - \mathbf{I})\mathbf{X}_{t-1}$  is stationary if and only if the rows of  $\mathbf{A} - \mathbf{I}$  are cointegrating vectors.
- ▶ One possibility is that  $(\mathbf{A} - \mathbf{I})$  has rank 0, in which case we do not have any cointegrating vectors.
- ▶ But if  $(\mathbf{A} - \mathbf{I})\mathbf{X}_{t-1}$  is stationary and the rank of  $\mathbf{A} - \mathbf{I}$  is  $R \geq 1$  then there are  $R$  cointegrating vectors (linearly independent combinations of the components of stationary processes).

## Cointegration: Johansen 4

- ▶ If  $R = 0$ , then there are no cointegrating vectors.
- ▶ If  $1 \leq R \leq n - 1$ , then there are  $R$  linearly independent cointegrating vectors.
- ▶ The Johansen method associates cointegration with determining the rank of  $\mathbf{A} - \lambda \mathbf{I}$  or equivalently the number of eigenvalues of  $\mathbf{A} - \lambda \mathbf{I}$  that are non-zero.
- ▶ These vectors are not uniquely determined, but they span the space of cointegrating vectors.

## Cointegration: Johansen 5

- ▶ Thus determining whether two or more processes are cointegrated using the Johansen approach assuming a VAR(1) process comes down to determining the rank of the  $\mathbf{A} - \mathbf{I}$ . Cointegrating vectors can be found as a basis for this matrix.
- ▶ In multivariate statistical analysis, there are two traditional ways to test for the rank of a matrix:  $H_0 R = 0$  vs  $R > 0$ . These involve converting a test of the rank into a test of the eigenvalues:
  - ▶ The maximum eigenvalue:  $H_0 : \lambda_{max} = 0$  vs  $\lambda_{max} > 0$ .
  - ▶ The trace of the matrix:  $Tr(\mathbf{A} - \mathbf{I}) = \sum_{i=1}^n = 0$  vs  $Tr(\mathbf{A} - \mathbf{I}) > 0$ .
- ▶ Note, there is a series of tests to find all the cointegrating vectors and the rank. For example, in the maximum eigenvalue test, if it is rejected, then there is a second test, whether the second largest eigenvalue is 0, and so on.



## Cointegration: Johansen 6

The analysis for VAR(1) processes extends in a straightforward way to VAR(p) processes

$$\mathbf{X}_t = \mathbf{a} + \sum_{i=1}^p \mathbf{A}_i \mathbf{X}_{t-i} + \boldsymbol{\epsilon}_t.$$

- ▶ Again, we assume that  $\mathbf{X}_t$  is  $\mathcal{I}(1)$ . If not, then the vector needs to be differenced enough times to bring it to  $\mathcal{I}(1)$ .
- ▶ Subtract  $\mathbf{X}_{t-1}$  from both sides to find:  $\mathbf{v}$

## Cointegration: Johansen 7

$$\begin{aligned}\nabla \mathbf{X}_t &= \mathbf{a} + (\mathbf{A}_1 - \mathbf{I})\mathbf{X}_{t-1} + \sum_{i=2}^p \mathbf{A}_i \mathbf{X}_{t-i} + \epsilon_t \\ &= \mathbf{a} + (\mathbf{A}_1 - \mathbf{I})\nabla \mathbf{X}_{t-1} + (\mathbf{A}_1 + \mathbf{A}_2 - \mathbf{I})\mathbf{X}_{t-2} \\ &\quad + \sum_{i=3}^p \mathbf{A}_i \mathbf{X}_{t-i} + \epsilon_t\end{aligned}$$

- ▶ This continues sequentially.
- ▶ Let

$$\mathbf{B}_i = (\mathbf{A}_1 + \cdots + \mathbf{A}_i - \mathbf{I}),$$

then subtracting and adding  $(\sum_{j=1}^k \mathbf{A}_j - \mathbf{I})$  on the right side yields:

## Cointegration: Johansen 8

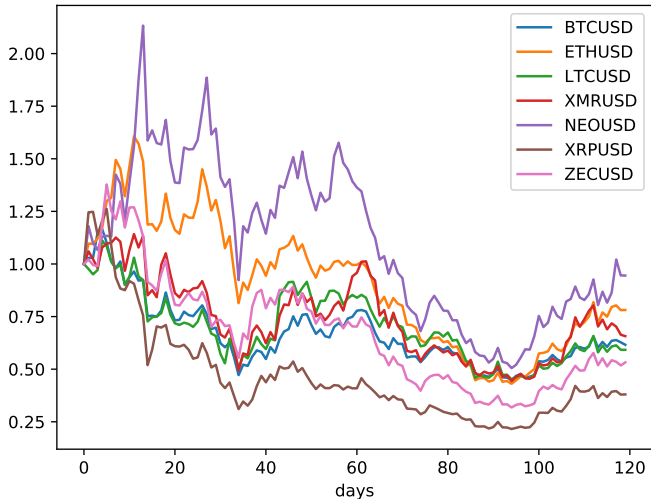
$$\nabla \mathbf{X}_t = \mathbf{a} + \sum_{j=1}^{p-1} \mathbf{B}_j \nabla \mathbf{X}_{t-j} + \mathbf{B}_p \mathbf{X}_{t-p} + \epsilon_t.$$

- ▶ Since we assumed  $\mathbf{X}_t$  is  $\mathcal{I}(1)$ , all terms involving  $\nabla \mathbf{X}_{t-j}$  are stationary, hence  $\mathbf{B}_p \mathbf{X}_p$  must be stationary.
- ▶ The term  $\mathbf{B}_p \mathbf{X}_{t-p}$  must be consistent with stationarity, so the arguments given for the  $p = 1$  case applied to  $(\mathbf{A} - \mathbf{I})$  now must be applied to  $\mathbf{B}_p$ .
- ▶ It's rank determines the number of cointegrating vectors, and the identity of those vectors.



# Cryptocurrency

Performance of cryptocurrencies



## Example: 1

```
import quandl
import pandas as pd
import matplotlib.pyplot as plt
import requests
import statsmodels.tsa.stattools as ts
from statsmodels.tsa.vector_ar.vecm import coint_johansen
def get_bitfinex_asset(asset, ts_ms_start, ts_ms_end):
    url = 'https://api.bitfinex.com/v2/candles/trade:1D:t' + asset +
    '/hist'
    params = 'start': ts_ms_start, 'end': ts_ms_end, 'sort': 1
    r = requests.get(url, params = params)
    data = r.json()
    return pd.DataFrame(data)[2]
```

## Example: 2

```
start_date = 1514768400000 # 1 January 2018, 00:00:00
end_date = 1527811199000 # 31 May 2018, 23:59:59
assets = ['BTCUSD', 'ETHUSD', 'LTCUSD', 'XMRUSD', 'NEOUSD',
          'XRPUSD', 'ZECUSD']
#Bitcoin, Ethereum, Litecoin, Monero, NEO, Ripple XRP, Zcash
crypto_prices = pd.DataFrame()
for a in assets:
    print('Downloading ' + a)
    crypto_prices[a] = get_bitfinex_asset(asset = a, ts_ms_start=
    start_date, ts_ms_end = end_date)
crypto_prices.head()
```

## Example: 3

```
# Normalize prices by first value
norm_prices = crypto_prices.divide(crypto_prices.iloc[0])
plt.figure(figsize = (15, 10))
plt.plot(norm_prices)
plt.xlabel('days')
plt.title('Performance of cryptocurrencies')
plt.legend(assets)
plt.show()
for a1 in crypto_prices.columns:
    for a2 in crypto_prices.columns:
        if a1 != a2:
            test_result = ts.coint(crypto_prices[a1], crypto_prices[a2])
            print(a1 + ' and ' + a2 + ': p-value = ' +
str(test_result[1]))
```



## Example: 4

BTCUSD and ETHUSD: p-value = 0.0657697980426905

BTCUSD and LTCUSD: p-value = 0.07347140678451415

BTCUSD and XMRUSD: p-value = 0.02157088942418218

BTCUSD and NEOUSD: p-value = 0.10239483419041967

BTCUSD and XRPUSD: p-value = 0.00900122457399112

BTCUSD and ZECUSD: p-value = 0.16378128244807383

ETHUSD and LTCUSD: p-value = 0.6090758251850134

ETHUSD and XMRUSD: p-value = 0.17284643088427865

ETHUSD and NEOUSD: p-value = 0.1287696772206111

ETHUSD and XRPUSD: p-value = 0.8353724771161406

ETHUSD and ZECUSD: p-value = 0.008516903095807193

## Example: 5

LTCUSD and XMRUSD: p-value = 0.17400005498761367

LTCUSD and NEOUSD: p-value = 0.1249504285029151

LTCUSD and XRPUSD: p-value = 0.34798612459501893

LTCUSD and ZECUSD: p-value = 0.2208460047208785

XMRUSD and NEOUSD: p-value = 0.15900072498514933

XMRUSD and XRPUSD: p-value = 0.10535218181116529

XMRUSD and ZECUSD: p-value = 0.08687381831452784

NEOUSD and XRPUSD: p-value = 0.7028115231545062

NEOUSD and ZECUSD: p-value = 0.19217634013579027

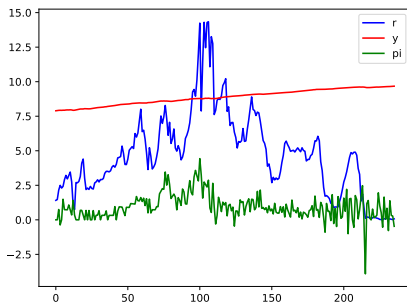
XRPUSD and ZECUSD: p-value = 0.6935781798945646

## Example: 6

The data set contains three time series,  $r$  (the 91-day T-bill rate),  $y$  (the log of real GDP), and  $\pi$  (the inflation rate). The goal is to perform an analysis to determine if any of these series are cointegrated.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from arch.unitroot import engle_granger
from arch.unitroot.cointegration import phillips_perron
from statsmodels.tsa.vector_ar.vecm import coint_johansen
data = pd.read.csv('TbGdpPi.csv')
```

## Example: 7



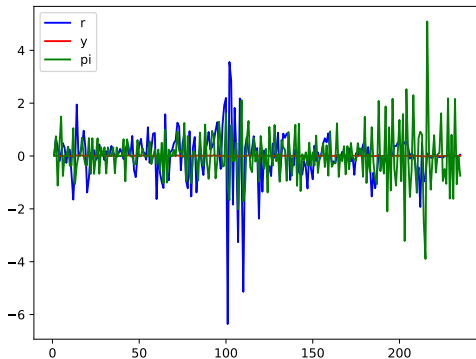
```
plt.plot(data['r'], 'b-', label = 'r')  
plt.plot(data['y'], 'r-', label = 'y')  
plt.plot(data['pi'], 'g-', label = 'pi')  
plt.legend(loc = 'best')  
plt.show()
```

## Example: 8

- ▶ `sm.tsa.stattools.adfuller(data['y'])`  
(-1.45, 0.56, 2, 233, '1%': -3.46, '5%': -2.87, '10%': -2.57,)
- ▶ `sm.tsa.stattools.adfuller(data['r'])`  
(-1.58, 0.49, 12, 223, '1%': -3.46, '5%': -2.87, '10%': -2.57,)
- ▶ `sm.tsa.stattools.adfuller(data['pi'])`  
(-1.78, 0.39, 15, 220, '1%': -3.46, '5%': -2.87, '10%': -2.57,)
- ▶ All are nonstationary and  $H_0$  (unit root) is clearly not rejected.
- ▶ `diffdata = data.diff()[1:]`

## Example: 9

```
plt.plot(data_diff['r'], 'b-', label = 'r')  
plt.plot(data_diff['y'], 'r-', label = 'y')  
plt.plot(data_diff['pi'], 'g-', label = 'pi')  
plt.legend(loc = 'best')  
plt.show()
```



## Example: 10

```
sm.tsa.stattools.adfuller(diffdata['y'])  
(-7.87, 4.89e-12, 1, 233, '1%': -3.46, '5%': -2.87, '10%':  
-2.57,)  
sm.tsa.stattools.adfuller(diffdata['r'])  
(-5.35, 4.25e-06, 11, 223, '1%': -3.46, '5%': -2.87, '10%':  
-2.57)  
sm.tsa.stattools.adfuller(diffdata['pi'])  
(-5.98, 1.87e-07, 14, 220, '1%': -3.46, '5%': -2.87, '10%':  
-2.57)
```

All three  $H_0$  are rejected, so all are  $\mathcal{I}(1)$

```
eg_test = engle_granger(data['r'], data['pi'], trend = 'n')  
eg_test
```

## Example: 11

Engle-Granger Cointegration Test (r, pi)

Statistic: -2.377

P-value: 0.119

Null: No Cointegration, Alternative: Cointegration

ADF Lag length: 3

Trend: c

Estimated Root: 0.856

```
eg_testry = engle_granger(data['r'],data['y'])
```

```
eg_testry
```

Engle-Granger Cointegration Test (r,y)

Statistic: -2.29

P-value: 0.378

Null: No Cointegration, Alternative: Cointegration

ADF Lag length: 3

Trend: c



## Example: 12

```
po_zt_testrpi = phillips_ouliaris(data['r'],data['pi'], trend = 'n',  
test_type = 'Zt')
```

```
po_zt_testrpi.summary()
```

Phillips-Ouliaris zt Cointegration Test

```
=====
```

|                |          |
|----------------|----------|
| Test Statistic | -11.755  |
| P-value        | 0.000    |
| Kernel         | Bartlett |
| Bandwidth      | 12.323   |

---

Trend: Constant

Critical Values: -2.48 (10%), -2.79 (5%), -3.39 (1%)

Null Hypothesis: No Cointegration

Alternative Hypothesis: Cointegration

Distribution Order: 2

## Example: 13

```
po_zt_testry = phillips_ouliaris(data['r'],data['y'],trend =  
'n',test_type = 'Zt')
```

```
po_zt_testry.summary()
```

Phillips-Ouliaris zt Cointegration Test

```
=====
```

|                |          |
|----------------|----------|
| Test Statistic | -2.051   |
| P-value        | 40.217   |
| Kernel         | Bartlett |
| Bandwidth      | 4.313    |

---

Trend: Constant

Critical Values: -2.48 (10%), -2.79 (5%), -3.39 (1%)

Null Hypothesis: No Cointegration

Alternative Hypothesis: Cointegration

Distribution Order: 2

## Example: 14

```
po_zt_testpiy = phillips_ouliaris(data['pi'],data['y'], trend =  
'n',test_type = 'zt')
```

```
po_zt_testpiy.summary()
```

Phillips-Ouliaris zt Cointegration Test

```
=====
```

|                |          |
|----------------|----------|
| Test Statistic | -11.880  |
| P-value        | 0.000    |
| Kernel         | Bartlett |
| Bandwidth      | 11.959   |

---

Trend: Constant

Critical Values: -2.48 (10%), -2.79 (5%), -3.39 (1%)

Null Hypothesis: No Cointegration

Alternative Hypothesis: Cointegration

Distribution Order: 2

## Example: 15

```
johansen_rst=coint_johansen(data,det_order=0, k_ar_diff=0)
johansen_rst.lr1
#tracestatistic
([131.27584764, 10.78357386, 4.18281327])
johansen_rst.lr2 # max eigenvalue statistic
([120.49227378, 6.60076059, 4.18281327])
johansen_rst.Results()
johansen_rst.cvt, critical values (.90,.95,.99) trace
((27.0669, 29.7961, 35.4628),
(13.4294, 15.4943, 19.9349),
( 2.7055, 3.8415, 6.6349))
johansen_rst.cvm critical values (.90,.95,.99) max eigenvalue
((18.8928, 21.1314, 25.865 ),
(12.2971, 14.2639, 18.52 ),
( 2.7055, 3.8415, 6.6349))
```

## Example: 16

```
trace_rst = 'Reject': list(johansen_rst.lr1 >  
johansen_rst.cvt[:,1])  
trace_rst = pd.DataFrame(trace_rst)  
trace_rst.index = ['r', 'y', 'pi']  
trace_rst
```

## Example: 17

```
max_eigv_rst  
= 'Reject': list(johansen_rst.lr2 > johansen_rst.cvm[:,1])  
max_eigv_rst = pd.DataFrame(max_eigv_rst)  
max_eigv_rst.index = ['r', 'y', 'pi']  
max_eigv_rst
```

# State Space Modeling: 1

- ▶ State space modeling Has a long history and is connected to important technology problems (e.g. Hidden Markov Models, which have been central in speech and image recognition)
- ▶ The idea is that there is a stochastic process which described the true state of a system (the state model),  $\{\mathbf{X}_t, t \geq 0\}$ .
- ▶ There is a second time series that represents the observation of this state process (the observation process),  $\{\mathbf{Y}_t, t \geq 0\}$ . These can be thought of as a noisy version of the true state process.
- ▶ The models can be univariate or multivariate, and are parameterized like the following:

$$\mathbf{X}_{t+1} = \mathbf{F}_t \mathbf{X}_t + \mathbf{V}_t,$$

$$\mathbf{Y}_t = \mathbf{G}_t \mathbf{X}_t + \mathbf{W}_t,$$

## State Space Models: 2

- ▶ The two noise processes,  $\mathbf{V}_t$  and  $\mathbf{W}_t$  are assumed to be  $WN(\mathbf{0}, \boldsymbol{\Sigma}_{X,t})$  and  $WN(\mathbf{0}, \boldsymbol{\Sigma}_{Y,t})$  respectively.
- ▶ The two noise processes are uncorrelated, i.e.  $E(\mathbf{V}_s \mathbf{W}_t) = \mathbf{0}, \forall s, t$ .
- ▶ Here is a simple example - writing an AR(1) model in state-space form (setting  $E(\mathbf{X}_t) = 0$ ):

$$X_t = \phi_1 X_{t-1} + \epsilon_{X,t},$$

$$X_{t-1} = X_{t-1},$$

$$Y_t = X_t + \epsilon_{Y,t}.$$



## State Space Models: 3

- ▶ The state space modeling form is very flexible and allows for the expression of a wide range of models. There are also three distinct types of problems that arise in this context, each solved by **Kalman Recursions: prediction/forecasting, filtering, and smoothing.**
- ▶ Suppose there is an unknown, latent state vector process  $\mathbf{X}_t$ . We have noisy observations of this state vector,  $\mathbf{Y}_t$  and some initial state  $Y_0$ . We want to use the observations,  $\mathbf{Y}_t$ , to make some inference about  $\mathbf{X}_t$  of the following type:

## State Space Models: 4

### ► Prediction or Forecasting

Estimate  $\mathbf{X}_{t+1}$  or  $\mathbf{X}_{t+k}$  using  $\mathbf{Y}_0, \mathbf{Y}_1, \dots, \mathbf{Y}_t$ . This can be denoted  $\hat{\mathbf{X}}_{(t+k)|t}$

### ► Filtering

Estimate  $\mathbf{X}_t$  using  $\mathbf{Y}_0, \mathbf{Y}_1, \dots, \mathbf{Y}_t$ . We denote this  $\hat{\mathbf{X}}_{t|t}$

### ► Smoothing

Estimate  $\mathbf{X}_t$  given  $\mathbf{Y}_1, \mathbf{Y}_2, \dots, \mathbf{Y}_T$ , for  $1 \leq t < T$ . This is denoted  $\hat{\mathbf{Y}}_{t|T}$  indicating the estimate of  $\mathbf{X}_t$  conditional on  $\mathcal{F}_T$ .

## State Space Models: 4

- ▶ In all three cases there are **Kalman recursions** to solve these problems based on minimizing mean square error. The recursions allow one to solve these problems “on-line,” that is in the case of forecasting and filtering to use the current results and the new data to obtain new estimates/forecasts without needing the entire data set that preceeded it.

## Kalman Prediction: 1

- ▶ In Kalman prediction, the goal is to predict/forecast  $\mathbf{X}_{t+1}$  given  $(\mathbf{Y}_0, \mathbf{Y}_1, \dots, \mathbf{Y}_t)$ , a future value of  $\mathbf{X}$  given the information from  $\mathbf{Y}$  up to time  $t$ .
- ▶ We let the mean squared error of the prediction be denoted by

$$\Omega_t = E((\mathbf{X}_{t+k} - \hat{\mathbf{X}}_{t+k})(\mathbf{X}_{t+k} - \hat{\mathbf{X}}_{t+k})^T).$$

- ▶ The Kalman prediction recursion begins with  $\hat{\mathbf{X}}_1$  and proceeds one-step at a time to  $\hat{\mathbf{X}}_{t+1}$ , as follows:

## Kalman Prediction: 2

$$\hat{\mathbf{X}}_1 = E(\mathbf{X}_1 | \mathbf{Y}_0),$$

$$\hat{\mathbf{X}}_{t+1} = \mathbf{F}_t \hat{\mathbf{X}}_t + \mathbf{\Theta} \mathbf{\Delta}_t^{-1} (\mathbf{Y}_t - \mathbf{G}_t \hat{\mathbf{X}}_t),$$

where

$$\mathbf{\Delta}_t = \mathbf{G}_t \mathbf{\Omega}_t \mathbf{G}_t^T + \mathbf{R}_t,$$

$$\mathbf{\Theta}_t = \mathbf{F}_t \mathbf{\Omega}_t \mathbf{G}_t^T.$$

## A Simple Example: 1

- ▶ Consider a simple linear regression model

$$Y_t = bZ_t + \epsilon_t,$$

where  $\epsilon_t$  are i.i.d.  $N(0, \sigma^2)$  and the predictor  $Z_t$  is known.

- ▶ We first express this as a state-space model with the goal being to estimate the unknown parameter  $b$ :

$$X_t = b,$$

$$Y_t = G_t X_t + W_t,$$

where  $G_t = Z_t$ ,  $R_t = \sigma^2$ ,  $F = 1$ ,  $Q = 0$

- ▶ Estimate  $X_t$  given  $Y_i$ ,  $i \leq t$ .

## A Simple Example: 2

- ▶ We know the answer from simple least squares

$$\hat{b}_t = \sum_{i=1}^t Y_i Z_i / \sum_{i=1}^t Z_i^2.$$

- ▶ How can this come from the Kalman equations, and how can  $\hat{b}_t$  be computed recursively?

$$\begin{aligned}\hat{b}_t &= \frac{\sum_{i=1}^{t-1} Z_i^2}{\sum_{i=1}^t Z_i^2} \frac{\sum_{i=1}^{t-1} Y_i Z_i + Y_t Z_t}{\sum_{i=1}^{t-1} Z_i^2} \\ &= \frac{\sum_{i=1}^{t-1} Z_i^2}{\sum_{i=1}^t Z_i^2} (\hat{b}_{t-1} + \frac{Y_t Z_t}{\sum_{i=1}^{t-1} Z_i^2}) \\ &= \text{next}\end{aligned}$$

## A Simple Example: 3

$$\begin{aligned}\hat{b}_t &= \hat{b}_{t-1} - \frac{Z_t^2}{\sum_{i=1}^t Z_i^2} \hat{b}_{t-1} + \frac{Y_t Z_t}{\sum_{i=1}^t Z_i^2} \\ &= \hat{b}_{t-1} + \frac{Z_t}{\sum_{i=1}^t Z_i^2} (Y_t - Z_t \hat{b}_{t-1})\end{aligned}$$

► The quantity  $K_t = Z_t / \sum_{i=1}^t Z_i^2$ , the “Kalman Gain.”



$$\text{Var}(\hat{b}_t) = (1 - K_t Z_t) \text{Var}(\hat{b}_{t-1}).$$



## Kalman Filtering

- ▶ For Kalman filtering, we want to predict/estimate the current state,  $\mathbf{X}_t$  given  $(\mathbf{Y}_0, \dots, \mathbf{Y}_t)$ . This is denoted by  $\mathbf{X}_{t|t}$ .
- ▶ The estimate is a combination of the one-step-ahead prediction from time  $t - 1$ ,  $\hat{\mathbf{X}}_t$  and the difference between the observed  $\mathbf{Y}_t$  and its predicted value using the observation equation,  $\mathbf{Y}_t - \mathbf{G}_t \hat{\mathbf{X}}_t$

$$\mathbf{X}_{t|t} = \hat{\mathbf{X}}_t + \mathbf{\Omega}_t \mathbf{G}_t^T \mathbf{\Delta}_t^{-1} (\mathbf{Y}_t - \mathbf{G}_t \hat{\mathbf{X}}_t),$$

where the relevant terms were defined earlier.